

Introduction to Deep Learning - A1 assignment

Deepak Somesh K J (s4587243)

Nallathambi Vethiappan (s4577663)

Divyanshi Singh (s4374827)

Abstract

This report presents experiments on neural network models for image classification and time prediction. Multilayer perceptrons (MLPs) and convolutional neural networks (CNNs) are implemented using TensorFlow/Keras and evaluated on the Fashion-MNIST and CIFAR-10 datasets. Various hyperparameter configurations are tested to identify models that generalize well across tasks. The “tell-the-time” problem is addressed using classification, regression, multi-head, and label representation. Results demonstrate that appropriate model design and tuning improve performance, with the best Fashion-MNIST models achieving over 92% accuracy and time-prediction models reaching errors below 10 minutes.

1 Introduction

The goal of this assignment is to develop practical skills in designing, training, and evaluating neural network models for image classification and regression tasks. Using the TensorFlow Keras library, multilayer perceptrons (MLPs) and convolutional neural networks (CNNs) are built and systematically compared on benchmark datasets including Fashion MNIST and CIFAR-10. Key hyperparameters such as learning rate, layer depth, and regularization are tuned to assess their impact on performance, following the methodologies and examples outlined in the textbook *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* [1].

The assignment also addresses the more complex task of implementing a CNN-based “tell-the-time” network that predicts times from images of analog clocks. This problem requires exploring different modeling approaches such as classification, regression, multi-head networks, and cyclical encoding of temporal data to accurately represent periodic time values. Performance is evaluated based on classification accuracy, mean absolute error, and domain-specific error measures designed to capture cyclical time discrepancies.

This report presents a comprehensive analysis of experimental results with quantitative metrics and visualisations, alongside critical reflections that link observed model behaviours to theoretical expectations. The findings highlight how architectural choices and hyperparam-

eter tuning influence model effectiveness on both standard and specialised tasks.

2 Task 1 - Exploring Neural Network Architectures for Image Classification

This task focuses on learning the Keras API for TensorFlow by implementing and experimenting with multilayer perceptron (MLP) and convolutional neural network (CNN) models. The experiments are conducted on the Fashion-MNIST and CIFAR-10 datasets to explore both grayscale and RGB image classification. Various hyperparameters are tested, including activation functions, optimisers, regularisation methods, and weight initialisers. Models are implemented using TensorFlow/Keras, and performance is evaluated mainly through validation and test accuracy. The objective is to gain practical experience with model design, tuning, and documentation rather than achieving the highest possible accuracy.

2.1 Dataset Description

This task uses two benchmark datasets for image classification: **Fashion-MNIST** and **CIFAR-10** [7, 5].

The **Fashion-MNIST** dataset consists of 70,000 grayscale images of size 28×28 pixels. It includes 60,000 training images and 10,000 test images. Each image belongs to one of 10 clothing categories, making it a more challenging alternative to classic MNIST dataset.

The **CIFAR-10** dataset contains 60,000 color images of size $32 \times 32 \times 3$ 10 categories representing objects such as animals and vehicles. It is divided into 50,000 training and 10,000 test images. The dataset’s variety and color make classification more complex than Fashion-MNIST.

Both datasets are standard benchmarks for evaluating image classification models. During training, a portion of the training data is held out as a validation set to monitor generalization and guide hyperparameter tuning.

2.2 Experimental Setup

The experiments were conducted using the TensorFlow and Keras frameworks. A fixed random seed was set for reproducibility. Training was performed on Google Colab using a T4 GPU for accelerated execution.

For each dataset, 10% of the training data was held out for validation to monitor model performance during training. The batch size was set to 128 for efficient computation.

All models were trained for up to 30 epochs. Early stopping was applied with a patience of six epochs to prevent overfitting. The learning rate was reduced by a factor of 0.5 if the validation loss did not improve for three epochs, with a minimum learning rate of 1×10^{-6} .

Validation and test accuracy were recorded to evaluate the effect of different model configurations.

2.3 Model Architectures and Hyperparameter Choices

Two neural network architectures were implemented: a **Multilayer Perceptron (MLP)** and a **Convolutional Neural Network (CNN)**. Both models were designed and trained using TensorFlow and Keras [3]. The architectural choices and hyperparameter configurations build upon standard examples and best practices outlined in the Keras example repository [4] and relevant literature [1].

MLP Architecture: The MLP takes flattened 28×28 grayscale images (784 features) as input. It consists of two or three fully connected layers with 128–512 units per layer. Each layer uses ReLU (default) or ELU activation and may include dropout for regularisation. The final layer uses softmax activation to classify 10 categories.

CNN Architecture: The CNN operates on $28 \times 28 \times 1$ images for Fashion-MNIST and $32 \times 32 \times 3$ images for CIFAR-10. It consists of two or three convolutional blocks with 3×3 or 5×5 filters (32–128 filters per layer). Each block includes batch normalisation and 2×2 max pooling. A dense layer of 128 units follows the convolutional layers, and a softmax output layer produces probabilities for 10 classes.

To systematically evaluate model behaviour, multiple hyperparameter combinations were tested. A total of 24 configurations were explored for each model type (MLP and CNN), resulting in 48 experiments on the Fashion-MNIST dataset. Table 1 summarises the MLP hyperparameters, while Table 2 summarises the CNN hyperparameters. These configurations vary hidden layers/filters,

activation functions, optimizers, weight initializers, regularization types, and dropout rates to analyse their effect on learning stability and accuracy.

Table 1: Hyperparameters explored for MLP models

Parameter	Values
Hidden Layers	[256,128], [512,256,128], [512,256]
Activation Func	ReLU (default), ELU
Optimizers	Adam, SGD, RMSprop
Initializers	Glorot Uniform, He Normal
Regularization	L1, L2, Dropout (0.2–0.4), None
Output Layer	Dense 10 with Softmax

Table 2: Hyperparameters explored for CNN models

Parameter	Values
Convolutional Layers	2–3 layers
Filters per Layer	32, 64, 128
Kernel Sizes	3×3 , 5×5
Activation Functions	ReLU (default), ELU
Batch Normalization	Applied after each conv layer
Pooling	MaxPooling 2×2
Dropout Rates	0.3–0.4
Optimizers	Adam, SGD, RMSprop
Initializers	Glorot Uniform, He Normal
Output Layer	Dense 10 with Softmax

All configurations were trained on the Fashion-MNIST dataset. The top-performing models were later retrained on CIFAR-10 to assess generalisation across datasets.

2.4 Experiment on Fashion-MNIST

2.4.1 Results

All 48 hyperparameter configurations (24 MLP and 24 CNN) were trained and evaluated on the Fashion-MNIST dataset. Figure 1 shows the validation accuracy for all configurations, providing a visual overview of comparative performance.

The top three CNN models based on validation accuracy are reported in Table 3. These models were selected for retraining on CIFAR-10 to assess generalization.

Table 3: Top three model configurations on Fashion-MNIST based on validation accuracy

Run ID	Test Accuracy
cnn_elu_rmsprop	
_glorot_uniform_None_64x128	92.7%
cnn_relu_adam	
_glorot_uniform_l2_32x64x128	92.0%
cnn_relu_sgd	
_glorot_uniform_None_32x64	92.9%

Glorot Uniform initialization. Differences in optimizer and regularization influenced final performance, highlighting the importance of dataset-specific tuning.

Overall, these experiments demonstrate that top-performing architectures on simpler datasets like Fashion-MNIST provide a useful baseline for more complex tasks, such as CIFAR-10 or future image-based problems like the “tell-the-time” task. The results highlight which architectural choices, regularization strategies, and optimizers generalize well, offering guidance for designing and tuning models in subsequent experiments.

3 Task 2 - Tell-the-time Dataset

The “tell-the-time” task focuses on developing CNN models to predict the time from analog clock images. It can be framed as either a multi-class classification problem (up to 720 classes) or a regression task predicting continuous time values. The study explores several approaches, including coarse and fine-grained classification, regression, multi-head networks for hours and minutes, and cyclical label encoding using sine-cosine representations. These methods examine how output representation affects performance, convergence, and interpretability. All models were implemented in TensorFlow/Keras, trained on synthetic clock datasets, and evaluated using accuracy and common-sense error (CSE) for classification, and mean absolute error (MAE) for regression-based models, considering the cyclical nature of time.

3.1 Dataset Description

The dataset consists of 18,000 synthetic grayscale images of analog clocks with associated labels indicating hours and minutes, stored in `labels.npy`. The images feature a variety of angles, rotations, and light reflections, making the task non-trivial. Two image resolutions are available: 75×75 pixels for initial experiments and 150×150 pixels for final evaluation. The data is split into 80% training, 10% validation, and 10% testing subsets with shuffling applied to eliminate ordering bias. Labels are formatted differently depending on the modelling approach—categorical classes for classification, continuous values for regression, dual outputs for multi-head networks, and sine-cosine encodings for cyclical label representations.

3.2 Common Experimental Setup

All models were implemented using TensorFlow/Keras and trained on Google Colab with a T4 GPU for accelerated execution. Preprocessing uniformly included normalisation of image pixel values to the $[0,1]$ range and reshaping images to include a single grayscale channel suitable for CNN input.

A consistent CNN backbone was employed for all models, comprising three convolutional layers each followed by batch normalisation, ReLU activation, max pooling, and dropout, with two fully connected layers at the end. Early stopping based on validation loss and ReduceLROnPlateau learning rate scheduling were utilized to prevent overfitting and ensure robust convergence in all models except the classification experiments.

The Adam optimiser was used, with learning rates fine-tuned per model to optimize performance. Evaluation metrics included accuracy and Common-Sense Error (CSE) for classification models, and Mean Absolute Error (MAE) alongside CSE for regression-based and cyclical models.

3.3 Model Architecture and Hyperparameters Choices

The CNN backbone consists of three convolutional layers with 32, 64, and 128 filters, designed to progressively capture increasingly complex spatial features appropriate for the input image sizes. Each convolutional layer uses a kernel size of 3×3 and is followed by a ReLU activation for effective nonlinear representation learning.

To stabilise training and mitigate overfitting, dropout with a rate of 0.25 is applied after each convolutional block, accompanied by batch normalisation. Max pooling layers reduce the spatial dimensions, controlling model complexity and computational load.

A batch size of 128 is generally employed to balance GPU workload and gradient estimation stability, except where specifically varied in some experiments. The base learning rate is set to 0.001, with adjustments made per experiment to optimise convergence.

These architectural choices and hyperparameter settings are consistent across the models implemented, ensuring comparability and robust performance.

3.4 Classification Model

The classification model predicts clock times from images using a CNN-based classifier. The effect of class granularity on prediction accuracy and CSE was investigated.

3.4.1 Training Process

Models used a softmax output layer with sparse categorical cross-entropy loss. Training ran for 60 epochs with a batch size of 128. Class granularity was varied among 12, 24, and 720 classes to explore performance trade-offs.

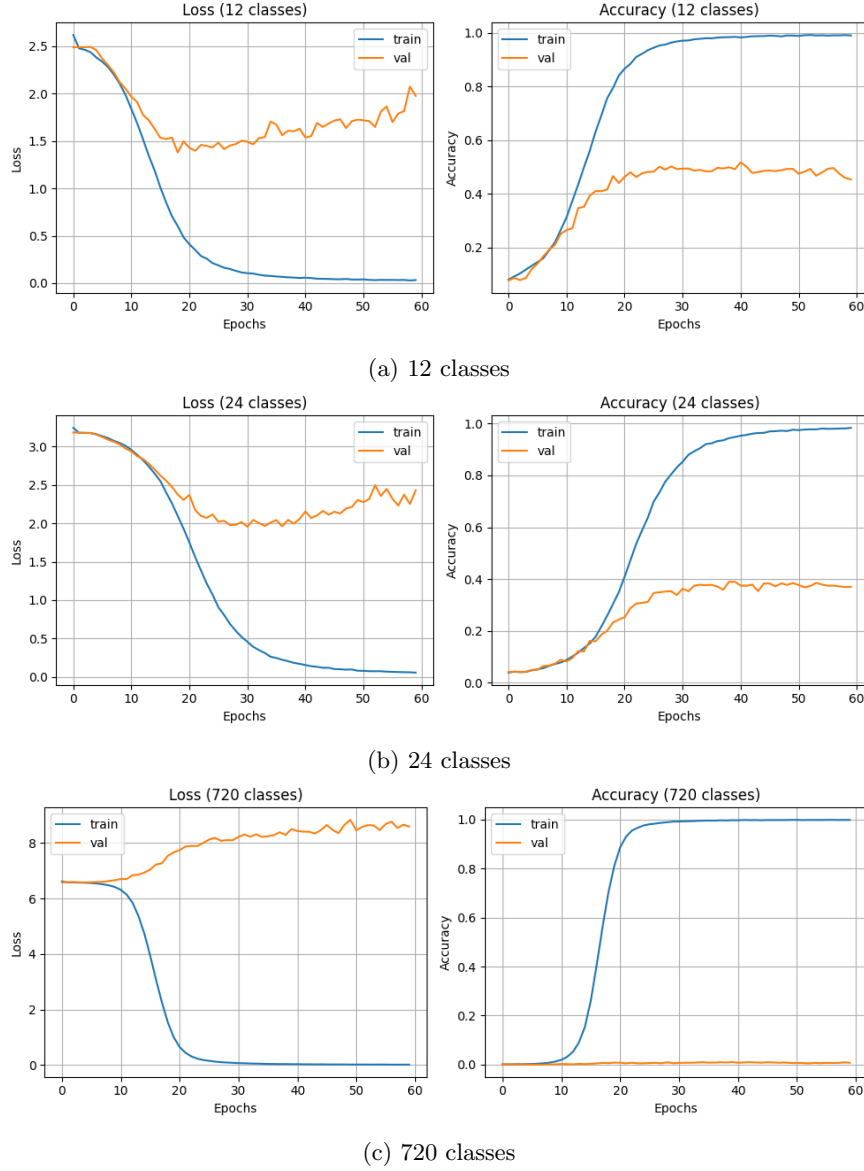


Figure 2: Training and validation curves for classification models

3.4.2 Results

The performance of the models is summarised in Table 5, using accuracy and common-sense error (CSE), which measures the mean circular difference in minutes between predicted and true clock times.

Table 5: Classification performance summary (150X150)

Classes	Accuracy	CSE (mins)
12	0.4761	63.87
24	0.3917	82.82
720	0.0139	182.93

Figure 2 shows the training and validation loss and accuracy curves for each class granularity (12, 24, and 720 classes).

3.4.3 Discussion

The classification results on 150×150 images reveal a trade-off between temporal granularity and prediction accuracy. The 12-class model achieves the best test accuracy of 47.61% with a common-sense error (CSE) of 63.87 minutes (Table 5). Figure 2a shows training accuracy reaching nearly 100% by epoch 60, while validation accuracy levels off around 47%. Validation loss stabilises near 1.5, indicating mild overfitting but good generalisation.

The 24-class model performs worse, with 39.17% accuracy and 82.82 minutes CSE (Figure 2b). Validation accuracy drops to 39% and loss fluctuates near 2.0, showing increased task difficulty and moderate overfitting.

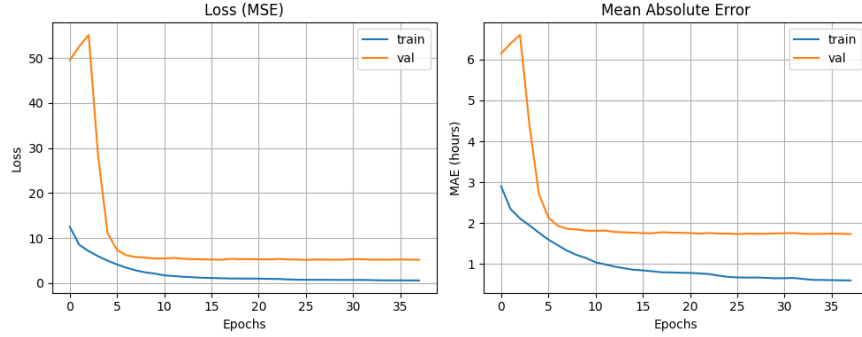


Figure 3: Training and validation loss/MAE curves for the regression model

For the 720-class model, performance declines sharply to 1.39% accuracy and 182.93 minutes CSE (Figure 2c). Validation accuracy remains near 1%, and loss rises above 8.0, reflecting severe overfitting and poor generalisation at minute-level granularity.

These results confirm that coarse classification effectively approximates time, while finer granularity suffers from boundary discontinuities (e.g., 11:59 to 12:00) and class imbalance, leading to high error and overfitting.

3.5 Regression Model

In the regression approach, the model predicts a continuous time value representing the clock reading in fractional hours (e.g., 3:30 as 3.5). This formulation treats time as a continuous variable rather than discrete categories, allowing the network to learn smoother transitions between adjacent time points.

3.5.1 Training Process

The output layer was a single linear neuron predicting fractional hours, trained using mean squared error (MSE) loss for 100 epochs with batch size 128.

3.5.2 Results

Model performance was evaluated using MAE (in hours) and Common-Sense Error (CSE, in minutes), which measures the circular difference between predicted and true times. The results are summarised in Table 6.

Table 6: Regression performance summary (150X150)

Metric	Value	Unit
MAE	1.9370	hours
CSE	114.16	minutes

Figure 3 illustrates the training and validation loss and MAE curves for the regression model. The model exhibits gradual convergence, although the validation loss plateaus after several epochs, suggesting limited fine-tuning around the optimal solution.

3.5.3 Discussion

The regression model on 150×150 images achieves a mean absolute error (MAE) of 1.94 hours (114.16 minutes). This indicates moderate performance in continuous time prediction. It performs better than the 720-class classification model. However, it is less accurate than the 12-class baseline.

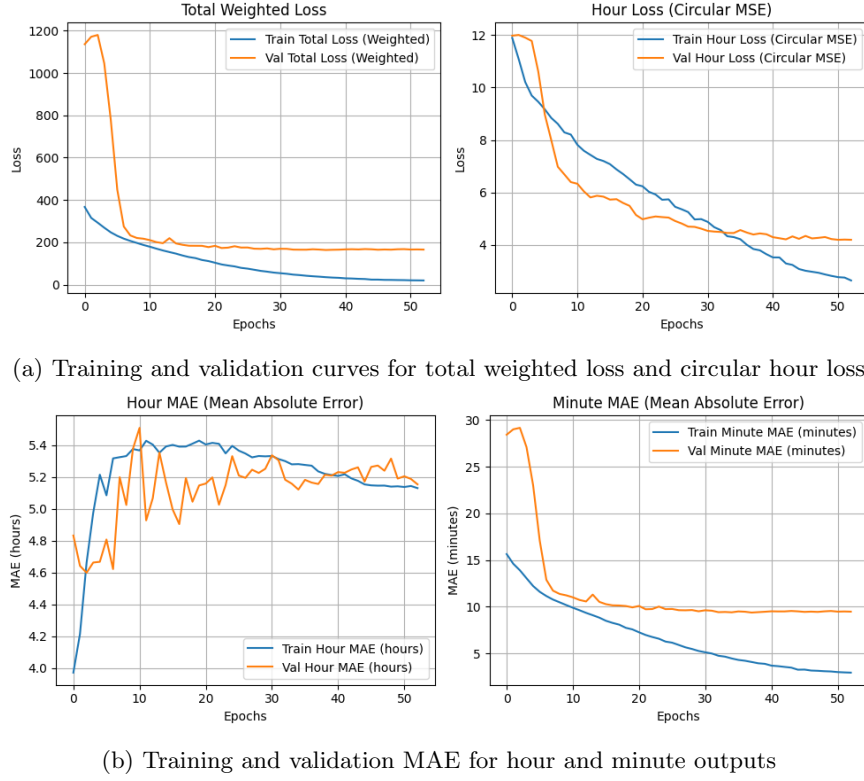
Figure 3 shows that both training and validation loss (MSE) decrease rapidly in the first 5 epochs. Training loss drops from above 10 to nearly zero. Validation loss falls from above 50 to about 5 and then plateaus. Validation MAE stabilizes near 1.7 hours after epoch 10. Training MAE continues to decrease and stays lower than validation.

The lower-than-expected performance was initially surprising. Analysis shows this is due to the cyclical nature of clock time. Small errors near wrap-around points, such as 11:59 to 12:00, are heavily penalized by the linear output.

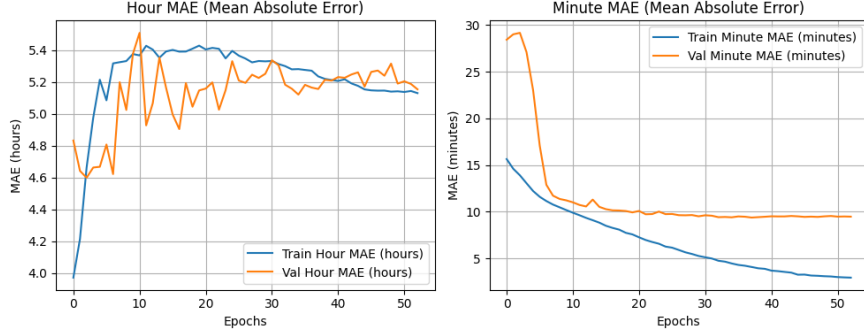
Regression provides smooth, continuous outputs and clean convergence during training. However, it lacks direct interpretability in discrete time units. This motivates the next experiments with multi-head and cyclical encoding models. They aim to better capture the periodic structure of time.

3.6 Multi-Head Models

The multi-head model predicts hours and minutes separately using two output heads, allowing the network to learn specialised representations for each component of time. Hours are treated with a circular MSE loss to account for the cyclical nature of clock hours, while minutes are optimised using standard MSE. This formulation allows better handling of clock wraparound points, such as 11:59 $\rightarrow$ 12:00, and improves interpretability over direct regression.



(a) Training and validation curves for total weighted loss and circular hour loss



(b) Training and validation MAE for hour and minute outputs

Figure 4: Training and validation behaviour of the multi-head model, illustrating separate error trends for hour and minute outputs and overall loss dynamics

3.6.1 Training Process

Two linear output heads predicted hours and minutes separately, using circular MSE for hours and standard MSE for minutes. Training proceeded for up to 150 epochs with a batch size of 128.

3.6.2 Results

Performance metrics include MAE for hours and minutes, as well as Common-Sense Error (CSE), which measures the circular difference in total minutes between predicted and true times. The results on the test set are summarised in Table 7.

Table 7: Multi-Head Model performance summary (150X150)

Metric	Value	Unit
Hour MAE	5.661	hours
Minute MAE	9.79	minutes
CSE	93	minutes

Figure 4 illustrates the training and validation curves for both loss and MAE. The model shows steady convergence, with validation curves closely following training curves, indicating improved generalisation over direct regression.

3.6.3 Discussion

The multi-head model improves interpretability and prediction accuracy compared to single-output regression. On the test set of 150X150 images, it achieves a mean absolute error (MAE) of 5.661 hours for hour prediction and 9.79 minutes for minute prediction. The overall common-sense error (CSE) is 93 minutes (Table 7).

Figure 4 shows the training and validation behaviour for each output head. The minute MAE drops quickly in the first few epochs and stabilises near 10 minutes. This demonstrates that minute-level prediction generalizes well and avoids significant overfitting. The hour MAE fluctuates more and settles above 5 hours, suggesting that hour estimation remains challenging and less consistent.

The total weighted loss and hour circular MSE curves also decrease steadily. The close alignment between the training and validation curves indicates stable optimisation and good generalisation. The use of circular MSE for hour prediction helps manage boundary transitions such as 11:59 to 12:00, reducing extreme hour errors.

Overall, the multi-head structure provides clearer outputs and smoother training compared to single-head regression. However, residual hour errors remain high, which shows that hour estimation is still difficult. The next set

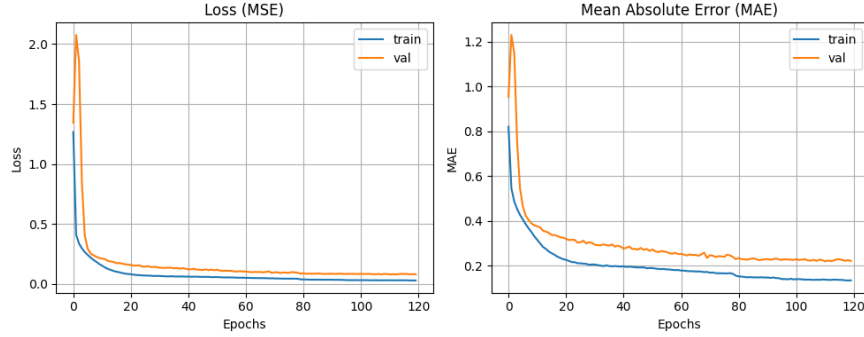


Figure 5: Training and validation curves of the cyclical encoding model, illustrating stable convergence and smooth generalisation behaviour

of experiments explores cyclical encoding methods to further reduce these errors and capture the periodic nature of time more effectively.

3.7 Label transformation

This model reformulates time prediction as a circular regression task by encoding clock times using sine and cosine transformations. Instead of predicting discrete hours or minutes directly, each label is represented as a point on the unit circle, defined by $\sin(\theta)$ and $\cos(\theta)$, where θ corresponds to the clock angle. This approach preserves the periodic nature of time and eliminates discontinuities at transitions such as 11:59 to 12:00 [2].

3.7.1 Training Process

A two-unit linear output layer predicted sine and cosine components of clock angles, trained with MSE loss for up to 120 epochs with batch size 128.

3.7.2 Results

Model performance is evaluated using two metrics: the linear MAE between sine-cosine vectors, and the Common-Sense Error (CSE), which computes the mean circular difference in minutes between predicted and true times. The test set results are summarised in Table 8.

Table 8: Cyclical encoding model performance summary (150X150)

Metric	Value	Unit
Linear MAE	0.2067	–
CSE	24.40	minutes

Figure 5 presents the training and validation loss (MSE) and mean absolute error (MAE) curves for this model.

3.7.3 Discussion

The sine-cosine encoding model on 150X150 images achieves a linear MAE of 0.2067 and a common-sense

error (CSE) of 24.40 minutes. This is the best performance among all tested models (Table 8). The large reduction in CSE shows that cyclical label representation effectively removes boundary discontinuities and systematic errors seen in regression and classification models.

Figure 5 shows that training and validation losses drop rapidly in the first few epochs. Both curves then converge smoothly to low, stable values. The validation curves plateau early without divergence or overfitting. This indicates that the model learns the periodic structure of time well and generalizes effectively to new images.

The model uses a simple output layer with only two linear nodes representing sine and cosine values. This improves circular consistency and reduces errors at time boundaries, such as the 11:59 to 12:00 transition. The geometric constraint ensures that predictions remain continuous and physically meaningful.

Overall, the cyclical encoding model provides a compact and theoretically sound solution for predicting continuous time. It retains regression interpretability, incorporates periodic awareness for accurate clock reading, and outperforms all previous models in both accuracy and CSE. This demonstrates the benefit of explicitly encoding temporal periodicity in vision-based time estimation.

3.8 Final Time-Prediction Model Optimisation

This model builds upon the label representation approach, reformulating time prediction as a circular regression task using sine and cosine transformations. Each clock time is represented as a point on the unit circle, defined by $\sin(\theta)$ and $\cos(\theta)$, preserving the periodic nature of time and avoiding discontinuities at hour transitions.

3.8.1 Motivation

Among the four models explored in previous experiments—classification, regression, multi-head, and sine-

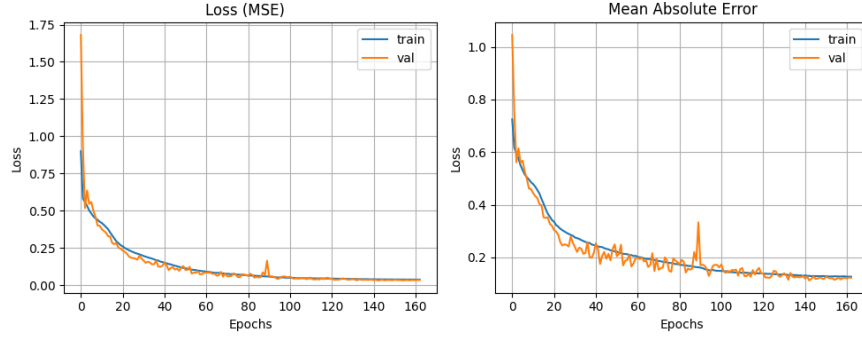


Figure 6: Training and validation curves of the optimised cyclical encoding model, demonstrating stable convergence and improved generalisation

cosine—the sine-cosine representation demonstrated superior performance, particularly in reducing boundary-related errors. Classification struggled with minute-level prediction, regression suffered from discontinuities at hour transitions, and the multi-head model showed only moderate improvements. By explicitly encoding temporal periodicity, the sine-cosine model ensures continuous, physically meaningful predictions, making it the most suitable choice for achieving sub-10-minute accuracy [6].

3.8.2 Training Process

The optimised sine-cosine model uses a deeper CNN with a Conv5 block followed by fully connected layers of 512 units. Dropout of 0.5 and L2 regularisation of $1e-4$ help prevent overfitting. Data augmentation includes rotation ($\pm 30^\circ$), translation ($\pm 10\%$), and zoom ($\pm 15\%$) to improve robustness to variations in clock images.

The model is trained using the Adam optimiser with a learning rate of 5×10^{-5} and batch size 64. Training runs for up to 250 epochs to allow more stable convergence than in previous experiment. Early stopping with patience 20 and ReduceLROnPlateau (factor 0.5, patience 8) are applied to prevent overfitting and adjust the learning rate dynamically.

The network predicts sine and cosine components using MSE loss. The new hyperparameters and augmentations improved convergence, stability, and generalisation compared to the previous version.

3.8.3 Results

Performance was evaluated using linear MAE between sine-cosine vectors and the Common-Sense Error (CSE) in minutes. Results on the test set are summarized in Table 9.

Table 9: Optimized cyclical encoding model performance summary

Metric	Value	Unit
Linear MAE	0.0946	—
CSE	8.52	minutes

Figure 6 shows training and validation MSE and MAE curves, illustrating smooth convergence and minimal overfitting.

3.8.4 Discussion

The optimised cyclical encoding model achieves a linear MAE of 0.0946 and a **common-sense error (CSE) of 8.52 minutes**, outperforming all previously tested architectures and meeting the target of sub-10-minute accuracy. This marked improvement confirms that deeper CNN architectures, combined with enhanced regularisation and carefully tailored training strategies, effectively capture the cyclical structure inherent in clock time.

Figure 6 illustrates smooth and stable convergence of both training and validation losses. MSE and MAE decline sharply in the first 20 epochs and continue steady improvement, with validation metrics closely tracking training, indicating strong generalisation and minimal overfitting despite increased model complexity.

This model benefits from a deeper Conv5 CNN backbone, deeper fully connected layers, higher dropout, L2 regularisation, and data augmentation strategies including rotation, translation, and zoom. Additionally, a reduced batch size and dynamic learning rate scheduling contribute to learning nuanced visual patterns and preserving the circular continuity of time via sine-cosine output encoding. This approach successfully handles challenging wrap-around transitions such as 11:59 to 12:00, producing physically meaningful and consistent predictions.

Compared to the original cyclical model, the optimised version more than halves both linear MAE and CSE, underscoring the impact of architectural depth and refined training procedures. It also significantly outperforms other models explored, including classification, regression, and multi-head networks, confirming its suitability for precise image-based clock time estimation.

In summary, the optimised cyclical encoding model presents a robust, interpretable, and theoretically grounded solution that explicitly models temporal periodicity and achieves state-of-the-art accuracy in this task.

4 Conclusion

This report presented experiments with neural network models for image classification and time prediction. The experiments used Fashion-MNIST and CIFAR-10 datasets. Convolutional neural networks performed better than multilayer perceptrons. The best CNN models achieved up to 92.9% accuracy on Fashion-MNIST. The CNNs also performed well on the more complex CIFAR-10 dataset.

The "tell-the-time" problem was studied using several modeling approaches. These included classification with different class granularities, regression, multi-head networks, and cyclical sine-cosine encoding. The sine-cosine encoding model gave the best results. It captured the periodic nature of time effectively. The optimized cyclical model reduced prediction error to below 10 minutes. This shows that careful design of model architecture and label representation can solve challenging time prediction tasks.

Overall, the results demonstrate the importance of matching model design to the task and data characteristics. The approach improves accuracy, generalization, and interpretability. This work provides useful insights for future image classification and time regression problems.

5 Contribution

- **Divyanshi:** Completed coding, experiments and reporting on Task 1 using Fashion-MNIST and CIFAR-10 datasets.
- **Nallathambi:** Worked on Task 2.1 - classification and regression models. Conducted experiments and prepared the corresponding report.
- **Deepak:** Developed Task 2.1 - multi-head models, label transformation using sine-cosine, and the optimized model for Task 2.2. Completed experiments and wrote the report sections.

References

- [1] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media, Sebastopol, CA, 2nd edition, 2019.
- [2] S Rao Jammalamadaka and A Sengupta. *Topics in Circular Statistics*. World Scientific, 2001.
- [3] Keras Team. Keras documentation. <https://keras.io/>, 2023. Accessed: 2025-11-07.
- [4] Keras Team. Keras examples repository. <https://github.com/keras-team/keras/tree/tf-keras/examples>, 2023. Accessed: 2025-11-07.
- [5] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [6] MLPills. Encoding cyclical features in time series. <https://mlpills.substack.com/p/encoding-cyclical-features-in-time-series>, 2025. Accessed: 2025-11-07.
- [7] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.